

KET Language Model on Penn Treebank

Training a Word-Level Language Model with Kan Extension Transformers

Simon Frost

Table of contents

Introduction	1
Setup	2
Data Loading	2
Batching	4
FunctorFlow Diagram	5
Neural Backend	5
Device Setup	6
Training Loop	7
Loss Curve	10
Evaluation	10
Categorical Interpretation	11
WikiText-2 Comparison	11
Summary	16

Introduction

In this vignette we train a word-level language model on the Penn Treebank (PTB) corpus using FunctorFlow’s KET (Kan Extension Transformer) architecture with Metal GPU acceleration on Apple Silicon.

The central idea is that attention—the core mechanism of modern transformers—can be understood as a left Kan extension $\Sigma_J(F)$: a universal way of aggregating information along a relation. In our language model:

Categorical concept	Neural interpretation
Source functor F	Token embeddings
Indexing functor J	Causal (autoregressive) mask

Categorical concept	Neural interpretation
Left Kan extension $\Sigma_J(F)$	Masked multi-head attention
Morphism decode	Linear projection to vocabulary
Cross-entropy loss	How well the diagram “commutes” with ground truth

We build the model as a FunctorFlow Diagram, compile it to a Lux neural network via `compile_to_lux`, and train it on PTB’s ~10K-word vocabulary. At the end we briefly compare results on WikiText-2.

Setup

```
using FunctorFlow
using Lux
using LuxCore
using Random
using Optimisers
using Zygote
using Statistics

# Lux layer types live in `FunctorFlowLuxExt` (FF v0.3.0+) and are not
# re-exported. Bring them into scope after `using Lux` triggers ext loading.
const _LuxExt = Base.get_extension(FunctorFlow, :FunctorFlowLuxExt)
const KETAttentionLayer = _LuxExt.KETAttentionLayer
const DiagramDenseLayer = _LuxExt.DiagramDenseLayer
```

Data Loading

The Penn Treebank dataset ships as one sentence per line with space-separated tokens. We load the raw tokens, insert `<eos>` at sentence boundaries, and build a vocabulary from the training split.

```
const DATA_DIR = joinpath(@__DIR__, "..", "..", "data")

function load_ptb(; split_name="train")
    path = joinpath(DATA_DIR, "ptb", "ptb.$(split_name).txt")
    tokens = String[]
    for line in eachline(path)
        stripped = strip(line)
```

```

        isempty(stripped) && continue
        append!(tokens, Base.split(stripped))
        push!(tokens, "<eos>")
    end
    return tokens
end

function build_vocab(tokens)
    unique_tokens = sort(unique(tokens))
    token_to_id = Dict{t => i for (i, t) in enumerate(unique_tokens)}
    return token_to_id, unique_tokens
end

function encode(tokens, vocab::Dict{String, Int})
    return [vocab[t] for t in tokens]
end

```

encode (generic function with 1 method)

```

train_tokens = load_ptb(split_name="train")
valid_tokens = load_ptb(split_name="valid")
test_tokens  = load_ptb(split_name="test")

vocab, id_to_token = build_vocab(train_tokens)
vocab_size = length(vocab)

println("Vocabulary size : $vocab_size")
println("Train tokens    : $(length(train_tokens))")
println("Valid tokens    : $(length(valid_tokens))")
println("Test tokens     : $(length(test_tokens))")

```

```

Vocabulary size : 10000
Train tokens    : 929589
Valid tokens    : 73760
Test tokens     : 82430

```

```

train_ids = encode(train_tokens, vocab)
valid_ids = encode(valid_tokens, vocab)
test_ids  = encode(test_tokens, vocab)

```

```
println("Sample IDs (first 10): ", train_ids[1:10])
println("Decoded back          : ", [id_to_token[i] for i in train_ids[1:10]])
```

Sample IDs (first 10): [238, 808, 951, 1326, 1477, 1692, 3774, 3921, 4068, 4381]

Decoded back : ["aer", "banknote", "berlitz", "calloway", "centrust", "cluett", "froms", "quebec"]

Batching

For next-token prediction we sample random windows of length `seq_len` from the tokenised corpus. The input is the first `seq_len` tokens and the target is the subsequent `seq_len` tokens (shifted by one position).

```
function make_batches(ids::Vector{Int}; seq_len=64, batch_size=32)
    n = length(ids)
    max_start = n - seq_len
    starts = rand(1:max_start, batch_size)
    inputs  = hcat([ids[s:s+seq_len-1] for s in starts]...) # seq_len × batch_size
    targets = hcat([ids[s+1:s+seq_len] for s in starts]...) # seq_len × batch_size
    return inputs, targets
end
```

`make_batches` (generic function with 1 method)

Quick sanity check:

```
inp, tgt = make_batches(train_ids; seq_len=8, batch_size=2)
println("Input shape : ", size(inp))
println("Target shape: ", size(tgt))
println("Input col 1 : ", [id_to_token[i] for i in inp[:, 1]])
println("Target col 1: ", [id_to_token[i] for i in tgt[:, 1]])
```

Input shape : (8, 2)

Target shape: (8, 2)

Input col 1 : ["helmut", "<unk>", "said", "<eos>", "he", "said", "the", "government"]

Target col 1: ["<unk>", "said", "<eos>", "he", "said", "the", "government", "will"]

FunctorFlow Diagram

We build the language model as a categorical diagram. The key components are:

1. Objects: Embeddings (the token embedding vectors), CausalRelation (the autoregressive mask), Contextualized (the KET output), and Logits (vocabulary-sized predictions).
2. Left Kan extension Σ : aggregates embeddings along the causal relation using learned multi-head attention—the core KET pattern.
3. Morphism decode: projects the contextualized representations to vocabulary logits.

```
D = Diagram(:KET_LM)

# Objects
add_object!(D, :Embeddings;      kind=:messages,
              description="Token embedding vectors (d_model × seq_len × batch)")
add_object!(D, :CausalRelation;   kind=:relation,
              description="Lower-triangular causal mask (seq_len × seq_len)")
add_object!(D, :Contextualized;   kind=:contextualized_messages,
              description="Context vectors after Kan extension")
add_object!(D, :Logits;           kind=:output,
              description="Vocabulary logits (vocab_size × seq_len × batch)")

# Left Kan extension  $\Sigma$ : the core KET attention pattern
 $\Sigma$ (D, :Embeddings;
      along   = :CausalRelation,
      name    = :aggregate,
      target  = :Contextualized,
      reducer = :ket_attention,
      description = "Aggregate embeddings via causal attention (left Kan extension)")

# Decode morphism: project to vocabulary
add_morphism!(D, :decode, :Contextualized, :Logits;
              description="Linear projection to vocabulary logits")

println(D)
```

Diagram :KET_LM {4 objects, 1 morphisms, 1 Kan, 0 losses}

Neural Backend

We now assign neural implementations to the diagram's operations. The embedding layer (integer IDs $\rightarrow$ dense vectors) sits outside the diagram as a preprocessing step; everything after that is

captured by the compiled Lux model.

```
const d_model = 64
const n_heads = 2
const seq_len = 32
const batch_size = 8

# Compile the diagram to a Lux model
ket_model = compile_to_lux(D;
    reducer_layers = Dict{:ket_attention ⇒ KETAttentionLayer(d_model; n_heads=n_heads)),
    morphism_layers = Dict{:decode ⇒ DiagramDenseLayer(d_model, vocab_size; name=:decode))
)

rng = Random.default_rng()
Random.seed!(rng, 42)

# Initialise embedding matrix manually (not part of the diagram)
embed_init = randn(rng, Float32, d_model, vocab_size) .* Float32(sqrt(2.0 / d_model))

# Initialise diagram model parameters and state
ket_ps, ket_st = Lux.setup(rng, ket_model)

println("Diagram model parameters: ", keys(ket_ps))
```

```
└ Warning: `replicate` doesn't work for `TaskLocalRNG`. Returning the same `TaskLocalRNG`.
└ @ LuxCore ~/.julia/packages/LuxCore/kQC9S/src/LuxCore.jl:18
Diagram model parameters: (:morph_decode, :red_ket_attention)
```

We combine all trainable parameters into a single NamedTuple so Zygote can differentiate through them in one pass:

```
ps_all = (embed = embed_init, ket = ket_ps)
println("Combined parameter groups: ", keys(ps_all))
```

```
Combined parameter groups: (:embed, :ket)
```

Device Setup

We use CPU for this tutorial. For Apple Silicon GPU acceleration, load `Metal.jl` and replace `dev = identity` with `dev = mtl` (see vignette 13 for a Metal example).

```

dev = identity # CPU; use `mtl` from Metal.jl for Apple Silicon GPU

ps_cpu = (embed = ps_all.embed, ket = ket_ps)
st_cpu = ket_st

println("Device: CPU")
println("Embedding type: ", typeof(ps_cpu.embed))

```

```

Device: CPU
Embedding type: Matrix{Float32}

```

Training Loop

We define a numerically stable cross-entropy loss and a full forward pass that goes from integer token IDs to loss.

```

# Numerically stable logsumexp
function logsumexp(x; dims)
    m = maximum(x; dims=dims)
    return m .+ log.(sum(exp.(x .- m); dims=dims))
end

function forward_and_loss(ket_model, ps, st, input_ids, target_ids, causal_mask)
    # 1. Embedding lookup: (d_model, vocab) × one-hot(seq_len × batch) → (d_model, seq_len, batch)
    sl, bs = size(input_ids)
    flat_ids = reshape(input_ids, :)
    emb_flat = ps.embed[:, flat_ids] # d_model × (seq_len*batch)
    embeddings = reshape(emb_flat, size(ps.embed, 1), sl, bs) # d_model × seq_len × batch

    # 2. Run the diagram: Σ (KET attention) + decode morphism
    diagram_inputs = Dict{
        :Embeddings    => embeddings,
        :CausalRelation => causal_mask
    }
    result, st_new = ket_model(diagram_inputs, ps.ket, st)
    logits = result[:values][:Logits] # vocab × seq_len × batch

    # 3. Cross-entropy loss
    log_probs = logits .- logsumexp(logits; dims=1) # vocab × seq_len × batch

    # Gather log-probs at target positions

```

```

v = size(logits, 1)
tgt_flat = reshape(target_ids, :)           # (seq_len*batch,)
lp_flat  = reshape(log_probs, v, :)        # vocab × (seq_len*batch)

# Build one-hot indicator matrix
onehot = Float32.(collect(1:v) .== reshape(tgt_flat, 1, :))

loss = -mean(sum(onehot .* lp_flat; dims=1))
return loss, st_new
end

```

Build the causal mask once (lower-triangular, so each position can attend only to itself and earlier positions):

```

using LinearAlgebra

causal_mask = Float32.(tril(ones(seq_len, seq_len)))
println("Causal mask shape: ", size(causal_mask))
println("Causal mask corner (5×5):")
display(causal_mask[1:5, 1:5])

```

Causal mask shape: (32, 32)

Causal mask corner (5×5):

```

5×5 Matrix{Float32}:
 1.0  0.0  0.0  0.0  0.0
 1.0  1.0  0.0  0.0  0.0
 1.0  1.0  1.0  0.0  0.0
 1.0  1.0  1.0  1.0  0.0
 1.0  1.0  1.0  1.0  1.0

```

Set up the optimiser and train for 50 steps:

```

opt = Optimisers.Adam(1.0f-3)
opt_state = Optimisers.setup(opt, ps_cpu)

n_steps = 50
log_interval = 10
losses = Float64[]

```



```

println("Starting training ($n_steps steps, seq_len=$seq_len, batch_size=$batch_size, d_model=$d_model)")
println("="^70)

for step in 1:n_steps
    # Sample a mini-batch
    inp, tgt = make_batches(train_ids; seq_len=seq_len, batch_size=batch_size)

    # Compute gradients
    (loss_val, st_new), grads = Zygote.withgradient(ps_cpu) do p
        forward_and_loss(ket_model, p, st_cpu, inp, tgt, causal_mask)
    end
    st_cpu = st_new

    # Update parameters
    opt_state, ps_cpu = Optimisers.update(opt_state, ps_cpu, grads[1])

    push!(losses, Float64(loss_val))

    if step % log_interval == 0 || step == 1
        ppl = exp(Float64(loss_val))
        println("Step $step / $n_steps | loss = $(round(loss_val; digits=4)) | perplexity = $(round(ppl; digits=4))")
    end
end

println("="^70)
println("Final training loss: $(round(losses[end]; digits=4))")
println("Final training perplexity: $(round(exp(losses[end]); digits=1))")

```

Starting training (50 steps, seq_len=32, batch_size=8, d_model=64)

```

=====
Step 1 / 50 | loss = 9.2036 | perplexity = 9932.8
Step 10 / 50 | loss = 9.1612 | perplexity = 9520.7
Step 20 / 50 | loss = 9.0432 | perplexity = 8460.8
Step 30 / 50 | loss = 8.7367 | perplexity = 6227.1
Step 40 / 50 | loss = 8.3088 | perplexity = 4059.3
Step 50 / 50 | loss = 7.7667 | perplexity = 2360.6
=====
Final training loss: 7.7667
Final training perplexity: 2360.6

```

Loss Curve

```
println("\nTraining loss curve (every $(log_interval) steps):\n")
for (i, step) in enumerate(log_interval:log_interval:n_steps)
    bar_len = max(0, round{Int, (maximum(losses) - losses[step]) / (maximum(losses) - minimum(losses))}, log_interval)
    bar = "█" ^ bar_len
    println("  Step $(lpad(step, 3)) | $(round(losses[step]; digits=3)) | $bar")
end
```

Training loss curve (every 10 steps):

Step	10	9.161	█
Step	20	9.043	██
Step	30	8.737	██████
Step	40	8.309	██████████
Step	50	7.767	████████████████

Evaluation

We estimate perplexity on the held-out validation set by averaging the cross-entropy loss over many random windows.

```
function estimate_perplexity(ket_model, ps, st, ids; seq_len=32, n_batches=10, batch_size=8)
    total_loss = 0.0
    cm = Float32.(tril(ones(seq_len, seq_len)))

    for _ in 1:n_batches
        inp, tgt = make_batches(ids; seq_len=seq_len, batch_size=batch_size)
        loss_val, _ = forward_and_loss(ket_model, ps, st, inp, tgt, cm)
        total_loss += Float64(loss_val)
    end

    avg_loss = total_loss / n_batches
    return exp(avg_loss), avg_loss
end

val_ppl, val_loss = estimate_perplexity(ket_model, ps_cpu, st_cpu, valid_ids)
println("Validation perplexity: $(round(val_ppl; digits=1))")
println("Validation loss      : $(round(val_loss; digits=4))")
```

Validation perplexity: 2007.5
Validation loss : 7.6047

Categorical Interpretation

What just happened, viewed through the lens of category theory?

The diagram as a functor. Our FunctorFlow diagram defines a functor $\mathcal{F}$ from a small indexing category (with objects `Embeddings`, `CausalRelation`, `Contextualized`, `Logits` and morphisms between them) into the category of vector spaces and linear maps. Training adjusts $\mathcal{F}$ so that the diagram’s predictions approximate the ground-truth next-token distribution.

The KET attention layer as a left Kan extension. The left Kan extension $\Sigma_J(F)$ is the universal construction that extends a functor F (token embeddings) along an indexing functor J (the causal mask). Concretely:

$$\Sigma_J(F)(t) = \operatorname{colim}_{s \leq t} \alpha(t, s) \cdot F(s)$$

where $\alpha(t, s)$ are the attention weights—learned as the softmax of scaled dot-product scores $QK^\top / \sqrt{d_k}$, masked by J so that $\alpha(t, s) = 0$ whenever $s > t$. This is exactly multi-head causal attention.

The causal mask as the indexing functor J . The lower-triangular mask defines which tokens can attend to which: position t can see positions $\{1, \dots, t\}$. This is a functor from the poset $(\mathbb{N}, \leq)$ into the category of relations.

The decode morphism. A natural transformation from the contextualized representation space to the vocabulary simplex—projecting the universal completion back to observable predictions.

Cross-entropy as an obstruction measure. The cross-entropy loss measures how far the diagram is from “commuting” with the ground truth distribution: a perfectly trained model would make the diagram commute exactly, with zero loss corresponding to zero obstruction.

WikiText-2 Comparison

We briefly repeat the experiment on WikiText-2 to compare perplexities across corpora using the same architecture.

```

function load_wikitext2(; split_name="train")
    path = joinpath(DATA_DIR, "wikitext-2", "wiki.$(split_name).tokens")
    tokens = String[]
    for line in eachline(path)
        stripped = strip(line)
        isempty(stripped) && continue
        words = Base.split(stripped)
        isempty(words) && continue
        append!(tokens, words)
        push!(tokens, "<eos>")
    end
    return tokens
end

wt2_train = load_wikitext2(split_name="train")
wt2_valid = load_wikitext2(split_name="valid")

```

216347-element Vector{String}:

```

"="
"Homarus"
"gammarus"
"="
"<eos>"
"Homarus"
"gammarus"
", "
"known"
"as"
:
"="
"="
"="
"Television"
"roles"
"="
"="
"="
"<eos>"

```

WikiText-2 has a larger vocabulary than PTB. We build a new vocabulary, re-initialise the model, and train briefly:

```

wt2_vocab, wt2_id_to_token = build_vocab(wt2_train)
wt2_vocab_size = length(wt2_vocab)

wt2_train_ids = encode(wt2_train, wt2_vocab)
wt2_valid_ids = encode(wt2_valid, wt2_vocab)

println("WikiText-2 vocabulary size: $wt2_vocab_size")
println("WikiText-2 train tokens   : $(length(wt2_train_ids))")
println("WikiText-2 valid tokens   : $(length(wt2_valid_ids))")

```

```

WikiText-2 vocabulary size: 33278
WikiText-2 train tokens   : 2075677
WikiText-2 valid tokens   : 216347

```

```

# Build a fresh model for the WikiText-2 vocabulary
wt2_diagram = Diagram(:KET_LM_WT2)
add_object!(wt2_diagram, :Embeddings;    kind=:messages)
add_object!(wt2_diagram, :CausalRelation; kind=:relation)
add_object!(wt2_diagram, :Contextualized; kind=:contextualized_messages)
add_object!(wt2_diagram, :Logits;        kind=:output)

Σ(wt2_diagram, :Embeddings;
  along=:CausalRelation, name=:aggregate,
  target=:Contextualized, reducer=:ket_attention)

add_morphism!(wt2_diagram, :decode, :Contextualized, :Logits)

wt2_model = compile_to_lux(wt2_diagram;
  reducer_layers = Dict{:ket_attention => KETAttentionLayer(d_model; n_heads=n_heads)},
  morphism_layers = Dict{:decode => DiagramDenseLayer(d_model, wt2_vocab_size; name=:decode)}
)

Random.seed!(rng, 42)
wt2_embed = randn(rng, Float32, d_model, wt2_vocab_size) .* Float32(sqrt(2.0 / d_model))
wt2_ket_ps, wt2_ket_st = Lux.setup(rng, wt2_model)

wt2_ps_cpu = (embed = wt2_embed, ket = wt2_ket_ps)
wt2_st_cpu = wt2_ket_st
wt2_opt_state = Optimisers.setup(Optimisers.Adam(1.0f-3), wt2_ps_cpu)

wt2_causal = Float32.(tril(ones(seq_len, seq_len)))

```

```
└ Warning: `replicate` doesn't work for `TaskLocalRNG`. Returning the same `TaskLocalRNG`.
└ @ LuxCore ~/.julia/packages/LuxCore/kQC9S/src/LuxCore.jl:18
```

32×32 Matrix{Float32}:

```
1.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  ...  0.0  0.0  0.0  0.0  0.0  0.0  0.0
1.0  1.0  0.0  0.0  0.0  0.0  0.0  0.0      0.0  0.0  0.0  0.0  0.0  0.0  0.0
1.0  1.0  1.0  0.0  0.0  0.0  0.0  0.0      0.0  0.0  0.0  0.0  0.0  0.0  0.0
1.0  1.0  1.0  1.0  0.0  0.0  0.0  0.0      0.0  0.0  0.0  0.0  0.0  0.0  0.0
1.0  1.0  1.0  1.0  1.0  0.0  0.0  0.0      0.0  0.0  0.0  0.0  0.0  0.0  0.0
1.0  1.0  1.0  1.0  1.0  1.0  0.0  0.0  ...  0.0  0.0  0.0  0.0  0.0  0.0  0.0
1.0  1.0  1.0  1.0  1.0  1.0  1.0  0.0      0.0  0.0  0.0  0.0  0.0  0.0  0.0
1.0  1.0  1.0  1.0  1.0  1.0  1.0  1.0      0.0  0.0  0.0  0.0  0.0  0.0  0.0
1.0  1.0  1.0  1.0  1.0  1.0  1.0  1.0      0.0  0.0  0.0  0.0  0.0  0.0  0.0
1.0  1.0  1.0  1.0  1.0  1.0  1.0  1.0      0.0  0.0  0.0  0.0  0.0  0.0  0.0
1.0  1.0  1.0  1.0  1.0  1.0  1.0  1.0      0.0  0.0  0.0  0.0  0.0  0.0  0.0
⋮      ⋮      ⋮      ⋮      ⋮      ⋮      ⋮      ⋮      ⋮      ⋮      ⋮      ⋮      ⋮      ⋮
1.0  1.0  1.0  1.0  1.0  1.0  1.0  1.0      0.0  0.0  0.0  0.0  0.0  0.0  0.0
1.0  1.0  1.0  1.0  1.0  1.0  1.0  1.0      0.0  0.0  0.0  0.0  0.0  0.0  0.0
1.0  1.0  1.0  1.0  1.0  1.0  1.0  1.0  ...  1.0  0.0  0.0  0.0  0.0  0.0  0.0
1.0  1.0  1.0  1.0  1.0  1.0  1.0  1.0      1.0  1.0  0.0  0.0  0.0  0.0  0.0
1.0  1.0  1.0  1.0  1.0  1.0  1.0  1.0      1.0  1.0  1.0  0.0  0.0  0.0  0.0
1.0  1.0  1.0  1.0  1.0  1.0  1.0  1.0      1.0  1.0  1.0  1.0  0.0  0.0  0.0
1.0  1.0  1.0  1.0  1.0  1.0  1.0  1.0      1.0  1.0  1.0  1.0  1.0  0.0  0.0
1.0  1.0  1.0  1.0  1.0  1.0  1.0  1.0  ...  1.0  1.0  1.0  1.0  1.0  1.0  0.0
1.0  1.0  1.0  1.0  1.0  1.0  1.0  1.0      1.0  1.0  1.0  1.0  1.0  1.0  1.0
```

```
wt2_n_steps = 50
```

```
println("Training KET-LM on WikiText-2 ($wt2_n_steps steps)")
println("=="^70)
```

```
for step in 1:wt2_n_steps
```

```
    inp, tgt = make_batches(wt2_train_ids; seq_len=seq_len, batch_size=batch_size)
```

```
    (loss_val, wt2_st_new), grads = Zygote.withgradient(wt2_ps_cpu) do p
        forward_and_loss(wt2_model, p, wt2_st_cpu, inp, tgt, wt2_causal)
    end
```

```
    wt2_st_cpu = wt2_st_new
    wt2_opt_state, wt2_ps_cpu = Optimisers.update(wt2_opt_state, wt2_ps_cpu, grads[1])
```

```
    if step % 10 == 0 || step == 1
```

```
        ppl = exp(Float64(loss_val))
```

```
        println("Step $step / $wt2_n_steps | loss = $(round(loss_val; digits=4)) | perplexity = $(round(ppl; digits=4))")
```

```
end
end
```

Training KET-LM on WikiText-2 (50 steps)

```
=====
Step 1 / 50 | loss = 10.4561 | perplexity = 34755.6
Step 10 / 50 | loss = 10.3604 | perplexity = 31584.2
Step 20 / 50 | loss = 10.2749 | perplexity = 28995.1
Step 30 / 50 | loss = 10.0973 | perplexity = 24276.5
Step 40 / 50 | loss = 9.8084 | perplexity = 18186.3
Step 50 / 50 | loss = 9.2471 | perplexity = 10374.0
```

```
wt2_val_ppl, wt2_val_loss = estimate_perplexity(wt2_model, wt2_ps_cpu, wt2_st_cpu, wt2_valid_i

println("\n" * "="^70)
println("Comparison (single KET block, d_model=$d_model, $n_heads heads, $n_steps steps):\n")
println(" Dataset      | Vocab | Val Loss | Val Perplexity")
println(" -----")
println(" PTB              | $(lpad(vocab_size, 5)) | $(lpad(round(val_loss; digits=3), 7)) | $
println(" WikiText-2       | $(lpad(wt2_vocab_size, 5)) | $(lpad(round(wt2_val_loss; digits=3),
println(" -----")
println("\nNote: These are intentionally small models (single KET block) trained briefly.")
println("State-of-the-art PTB perplexity is ~55; our goal is to demonstrate the KET pattern.")
```

```
=====
Comparison (single KET block, d_model=64, 2 heads, 50 steps):
```

Dataset	Vocab	Val Loss	Val Perplexity
PTB	10000	7.605	2007.5
WikiText-2	33278	8.815	6732.1

Note: These are intentionally small models (single KET block) trained briefly.
State-of-the-art PTB perplexity is ~55; our goal is to demonstrate the KET pattern.

Summary

We demonstrated how to build and train a word-level language model using FunctorFlow’s categorical abstractions:

1. Diagram construction — the model architecture is a categorical diagram with objects (embedding space, causal relation, contextualized space, logits) connected by a left Kan extension and a decode morphism.
2. Neural compilation — `compile_to_lux` turns the diagram into a differentiable Lux model, with `KETAttentionLayer` implementing the Kan extension as multi-head attention and `DiagramDenseLayer` implementing the decode morphism.
3. GPU training — `Metal.jl` moves all parameters and computation to Apple Silicon, while `Zygote` provides automatic differentiation through the entire pipeline.
4. Categorical semantics — the attention mechanism is not an ad hoc design choice but a principled instance of a universal construction (left Kan extension), and training minimises the obstruction to the diagram’s commutativity with ground truth.

This pattern generalises naturally: stacking multiple KET blocks gives a deep transformer, composing with DB squares adds consistency constraints, and swapping the causal mask for a graph adjacency matrix turns the language model into a graph transformer—all within the same categorical framework.